Counting inversions in an array

I'm designing an algorithm to do the following: Given array `A[1... n]` , for every `i < j` , find all inversion pairs such that `A[i] > A[j]` . I'm using merge sort and copying array A to array B and then comparing the two arrays, but I'm having a difficult time seeing how I can use this to find the number of inversions. Any hints or help would be greatly appreciated.

algorithm

edited Oct 25 '14 at 9:12
Martijn Pieters ♦
365k 36 706 899

asked Dec 3 '08 at 16:07
el diablo
1,211 1 10 17

If you're posting in the algorithms tag, start by explaining your solution in prose before pasting blocks of code. It's generally more convincing, as well as accessible to more people (there is code below in Java, Python, C#, C++, C, Scala). If you think and write clearly, it'll help your code too. – Colonel Panic Dec 24 '14 at 11:02

28 Answers

The only advice I could give to this (which looks suspiciously like a homework question ;)) is to first do it manually with a small set of numbers (e.g. 5), and then write down the steps you took to solve the problem.

This should allow you to figure out a generic solution you can use to write the code.

edited Oct 5 '12 at 4:19

community wiki
2 revs, 2 users 91%
Andrew Rollings

2 For information; the question is actually tagged as "homework". This question hasn't been edited yet so he must have tagged it as such when he submitted it. – Doctor Jones Dec 3 '08 at 16:33

6 Good point... And while I applaud students using SO as a resource to help with homework, I'm not sure that the solution should be spelled out directly for them to cut and paste, as I've seen in other questions. :) – Andrew Rollings Dec 3 '08 at 17:03

7 No, it shouldn't, but he should be given enough information to be able to work out the solution for himself. In other words, hints, not answers. – Lasse V. Karlsen Dec 3 '08 at 17:43

5 Which is hopefully what I did. – Andrew Rollings Dec 3 '08 at 17:46

1 Well, this is not a bad advice, but what if I can only figure out a O(n^2) algo? – Alcott Oct 25 '12 at 2:20

So here is O(n log n) solution in java.

```
long merge(int[] arr, int[] left, int[] right) {
    int i = 0, j = 0, count = 0;
    while (i < left.length || j < right.length) {
        if (i == left.length) {
            arr[i+j] = right[j];
            j++;
        } else if (j == right.length) {
            arr[i+j] = left[i];
            i++;
        } else if (left[i] <= right[j]) {
            arr[i+j] = left[i];
            i++;
        } else {
            arr[i+j] = right[j];
            count += left.length-i;
            j++;
```

```
            }
        }
        return count;
    }

    long invCount(int[] arr) {
        if (arr.length < 2)
            return 0;

        int m = (arr.length + 1) / 2;
        int left[] = Arrays.copyOfRange(arr, 0, m);
        int right[] = Arrays.copyOfRange(arr, m, arr.length);

        return invCount(left) + invCount(right) + merge(arr, left, right);
    }
```

This is almost normal merge sort, the whole magic is hidden in merge function. Note that while sorting algorithm remove inversions. While merging algorithm counts number of removed inversions (sorted out one might say).

The only moment when inversions are removed is when algorithm takes element from the right side of an array and merge it to the main array. The number of inversions removed by this operation is the number of elements left from the the left array to be merged. :)

Hope it's explanatory enough.

edited Aug 29 '14 at 9:48
barghest
3 2

answered Jun 21 '11 at 11:58
Marek Kirejczyk
1,733 1 9 7

4 This is such a cleaner method!!! – redDragonzz Sep 6 '11 at 7:48

2 I tried running this and I did not get the correct answer. Are you supposed to call invCount(intArray) inside main to get started? With the intArray being the unsorted array of int's? I ran it with a an array of many integers and got a -1887062008 as my answer. What am I doing wrong? – nearpoint Aug 26 '13 at 22:50

Here are results of the tests: link – Yuriy Chernyshov Oct 20 '13 at 17:36

2 +1, See similar solution in **C++11**, including a general iterator-based solution and sample random testbed using sequences from 5-25 elements. Enjoy!. – WhozCraig Apr 20 '14 at 20:15

+1 for the code is shorter and easy to understand – Jerky Jul 5 '14 at 23:03

I've found it in O(n * log n) time by the following method.

1. Merge sort array A and create a copy (array B)
2. Take A[1] and find its position in sorted array B via a binary search. The number of inversions for this element will be one less than the index number of its position in B since every lower number that appears after the first element of A will be an inversion.

   2a. accumulate the number of inversions to counter variable num_inversions.

   2b. remove A[1] from array A and also from its corresponding position in array B
3. rerun from step 2 until there are no more elements in A.

Here's an example run of this algorithm. Original array A = (6, 9, 1, 14, 8, 12, 3, 2)

1: Merge sort and copy to array B

B = (1, 2, 3, 6, 8, 9, 12, 14)

2: Take A[1] and binary search to find it in array B

A[1] = 6

B = (1, 2, 3, **6**, 8, 9, 12, 14)

6 is in the 4th position of array B, thus there are 3 inversions. We know this because 6 was in the first position in array A, thus any lower value element that subsequently appears in array A would have an index of j > i (since i in this case is 1).

2.b: Remove A[1] from array A and also from its corresponding position in array B (bold elements are removed).

A = (**6,** 9, 1, 14, 8, 12, 3, 2) = (9, 1, 14, 8, 12, 3, 2)

B = (1, 2, 3, **6,** 8, 9, 12, 14) = (1, 2, 3, 8, 9, 12, 14)

3: Rerun from step 2 on the new A and B arrays.

A[1] = 9

B = (1, 2, 3, 8, 9, 12, 14)

9 is now in the 5th position of array B, thus there are 4 inversions. We know this because 9 was

in the first position in array A, thus any lower value element that subsequently appears would have an index of j > i (since i in this case is again 1). Remove A[1] from array A and also from its corresponding position in array B (bold elements are removed)

A = (**9**, 1, 14, 8, 12, 3, 2) = (1, 14, 8, 12, 3, 2)

B = (1, 2, 3, 8, **9**, 12, 14) = (1, 2, 3, 8, 12, 14)

Continuing in this vein will give us the total number of inversions for array A once the loop is complete.

Step 1 (merge sort) would take O(n * log n) to execute. Step 2 would execute n times and at each execution would perform a binary search that takes O(log n) to run for a total of O(n * log n). Total running time would thus be O(n * log n) + O(n * log n) = O(n * log n).

Thanks for your help. Writing out the sample arrays on a piece of paper really helped to visualize the problem.

answered Dec 3 '08 at 18:36

el diablo
1,211 1 10 17

1 why use merge sort not quick sort? – Alcott Oct 12 '11 at 3:21

5 @Alcott Quick sort has worst running time of O(n^2), when the list is already sorted, and first pivot is chosen every round. Merge sort's worst case is O(n log n) – user482594 Feb 21 '12 at 7:09

21 The removal step from a standard array makes your algorithm O(n^2), due to shifting the values. (That's why insertion sort is O(n^2)) – Kyle Butt Oct 2 '12 at 18:49

starting with the first element of array B and counting the elements before it in array A would also give the same result, provided you eliminate them as you described in your answer. – tutak Jan 31 '13 at 17:51

@el diablo How to remove elements to avoid n^2 complexity?? – Jerky Jul 5 '14 at 20:22

In Python

```
# O(n log n)

def count_inversion(lst):
    return merge_count_inversion(lst)[1]

def merge_count_inversion(lst):
    if len(lst) <= 1:
        return lst, 0
    middle = int( len(lst) / 2 )
    left, a = merge_count_inversion(lst[:middle])
    right, b = merge_count_inversion(lst[middle:])
    result, c = merge_count_split_inversion(left, right)
    return result, (a + b + c)

def merge_count_split_inversion(left, right):
    result = []
    count = 0
    i, j = 0, 0
    left_len = len(left)
    while i < left_len and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            count += left_len - i
            j += 1
    result += left[i:]
    result += right[j:]
    return result, count


#test code
input_array_1 = []  #0
input_array_2 = [1] #0
input_array_3 = [1, 5]  #0
input_array_4 = [4, 1] #1
input_array_5 = [4, 1, 2, 3, 9] #3
input_array_6 = [4, 1, 3, 2, 9, 5]  #5
input_array_7 = [4, 1, 3, 2, 9, 1]  #8

print count_inversion(input_array_1)
print count_inversion(input_array_2)
print count_inversion(input_array_3)
print count_inversion(input_array_4)
print count_inversion(input_array_5)
print count_inversion(input_array_6)
print count_inversion(input_array_7)
```

answered Mar 1 '13 at 5:20

mkso
1,501 2 13 26

2 I'm baffled by how this managed to get to +13 - I'm not particularly skilled in Python, but it seems pretty much the same as the Java version presented 2 years before, except that this **doesn't provide any explanation whatsoever**. Posting answers in every other language is actively harmful IMO - there are probably thousands, if not many more, languages - I hope no-one will argue that we should be posting thousands of answers to a question - Stack Exchange wasn't made for that. – Dukeling Jun 6 '14 at 18:28

@Dukeling Your point may well be correct for all I know. What causes me to doubt it, though, is that you had to use the fallacy of false alternative to justify it. There may be thousands of languages but there are not thousands of extremely popular languages. There probably aren't even five ahead of Python. Now, you could have said "I hope no-one will argue that we should be posting five answers to a question"... – tennenrishin Jun 6 '14 at 22:04

@tennenrishin Okay, maybe not thousands. But where do we draw the line though? There are currently, as I count it, **ten** answers giving **the same approach** already. That's about **43% of the answers** (excl. the non-answer) - quite a bit of space to take up given that there are half a dozen other approaches presented here. Even if there are just 2 answers for the same approach, that still unnecessarily dilutes the answers. And I made a pretty decent argument for **this answer specifically** not being useful in my previous comment. – Dukeling Jun 6 '14 at 22:22

@Dukeling Like you, I'm unfamiliar with Python, and more familiar with Java. I find this solution much less readable than the Java one. It stands to reason then that for some people the converse could be true to the same extent. – tennenrishin Jun 10 '14 at 17:56

@tennenrishin I've very familiar with Java, yet I find **the high-level explanation** a hundred times more readable than the Java code. If the languages in the answers were swapped, my response would've likely been identical (but not if it were any old language or any old syntax in the first answer - both of these make use of very common syntax that should be readable by any decent programmer, the assumption being that any decent programmer would have learnt a language that has somewhat similar syntax). – Dukeling Jun 10 '14 at 18:14

I had a question similar to this for homework actually. I was restricted that it must have O(nlogn) efficiency.

I used the idea you proposed of using Mergesort, since it is already of the correct efficiency. I just inserted some code into the merging function that was basically: Whenever a number from the array on the right is being added to the output array, I add to the total number of inversions, the amount of numbers remaining in the left array.

This makes a lot of sense to me now that I've thought about it enough. Your counting how many times there is a greater number coming before any numbers.

hth.

answered Feb 12 '09 at 21:47
statistic

2 i support your answer, essential difference from merge sort is in merge function when element of 2nd right array gets copied to output array => increment inversion counter by number of elements remaining in 1st left array – Alex.Salnikov Aug 7 '12 at 17:52

Note that the answer by Geoffrey Irving is wrong.

> The number of inversions in an array is half the total distance elements must be moved in order to sort the array. Therefore, it can be computed by sorting the array, maintaining the resulting permutation p[i], and then computing the sum of abs(p[i]-i)/2. This takes O(n log n) time, which is optimal.
>
> An alternative method is given at http://mathworld.wolfram.com/PermutationInversion.html. This method is equivalent to the sum of max(0, p[i]-i), which is equal to the sum of abs(p[i]-i])/2 since the total distance elements move left is equal to the total distance elements move to the right.

Take the sequence { 3, 2, 1 } as an example. There are three inversions: (3, 2), (3, 1), (2, 1), so the inversion number is 3. However, according to the quoted method the answer would have been 2.

answered Jun 19 '12 at 0:16

user1465038
51 1 1

Check this out:
http://www.cs.jhu.edu/~xfliu/600.363_F03/hw_solution/solution1.pdf

I hope that it will give you the right answer.

- 2-3 Inversion part (d)

- It's running time is O(nlogn)

edited Jan 24 '12 at 13:30
mrnx
16.6k 5 23 38

answered Feb 11 '11 at 10:44

Hafsa
41 1

Here is one possible solution with variation of binary tree. It adds a field called rightSubTreeSize to each tree node. Keep on inserting number into binary tree in the order they appear in the array. If number goes lhs of node the inversion count for that element would be (1 + rightSubTreeSize). Since all those elements are greater than current element and they would have appeared earlier in the array. If element goes to rhs of a node, just increase its rightSubTreeSize. Following is the code.

```
Node {
    int data;
    Node* left, *right;
    int rightSubTreeSize;

    Node(int data) {
        rightSubTreeSize = 0;
    }
};

Node* root = null;
int totCnt = 0;
for(i = 0; i < n; ++i) {
    Node* p = new Node(a[i]);
    if(root == null) {
        root = p;
        continue;
    }

    Node* q = root;
    int curCnt = 0;
    while(q) {
        if(p->data <= q->data) {
            curCnt += 1 + q->rightSubTreeSize;
            if(q->left) {
                q = q->left;
            } else {
                q->left = p;
                break;
            }
        } else {
            q->rightSubTreeSize++;
            if(q->right) {
                q = q->right;
            } else {
                q->right = p;
                break;
            }
        }
    }

    totCnt += curCnt;
  }
  return totCnt;
```

answered Apr 17 '13 at 9:15

prasadvk
663 1 7 11

```
public static int mergeSort(int[] a, int p, int r)
{
    int countInversion = 0;
    if(p < r)
    {
        int q = (p + r)/2;
        countInversion = mergeSort(a, p, q);
        countInversion += mergeSort(a, q+1, r);
        countInversion += merge(a, p, q, r);
    }
    return countInversion;
}

public static int merge(int[] a, int p, int q, int r)
{
    //p=0, q=1, r=3
    int countingInversion = 0;
    int n1 = q-p+1;
    int n2 = r-q;
    int[] temp1 = new int[n1+1];
    int[] temp2 = new int[n2+1];
    for(int i=0; i<n1; i++) temp1[i] = a[p+i];
    for(int i=0; i<n2; i++) temp2[i] = a[q+1+i];

    temp1[n1] = Integer.MAX_VALUE;
    temp2[n2] = Integer.MAX_VALUE;
    int i = 0, j = 0;
```

```
    for(int k=p; k<=r; k++)
    {
        if(temp1[i] <= temp2[j])
        {
            a[k] = temp1[i];
            i++;
        }
        else
        {
            a[k] = temp2[j];
            j++;
            countingInversion=countingInversion+(n1-i);
        }
    }
    return countingInversion;
}
public static void main(String[] args)
{
    int[] a = {1, 20, 6, 4, 5};
    int countInversion = mergeSort(a, 0, a.length-1);
    System.out.println(countInversion);
}
```

answered Jul 24 '13 at 7:18

Trying
4,993 1 18 50

Is this much different from the Java and Python solutions posted already? Also, code-only answers are not particularly good IMO, especially considering this question didn't even specify a language. – Dukeling Jun 6 '14 at 17:40

I wonder why nobody mentioned binary-indexed trees yet. You can use one to maintain prefix sums on the values of your permutation elements. Then you can just proceed from right to left and count for every element the number of elements smaller than it to the right:

```
def count_inversions(a):
  res = 0
  counts = [0]*(len(a)+1)
  rank = { v : i+1 for i, v in enumerate(sorted(a)) }
  for x in reversed(a):
    i = rank[x] - 1
    while i:
      res += counts[i]
      i -= i & -i
    i = rank[x]
    while i <= len(a):
      counts[i] += 1
      i += i & -i
  return res
```

The complexity is O(n log n), and the constant factor is very low.

answered Apr 21 '14 at 16:40

Niklas B.
45.3k 5 88 136

Here is a C code for count inversions

```
#include <stdio.h>
#include <stdlib.h>

int  _mergeSort(int arr[], int temp[], int left, int right);
int merge(int arr[], int temp[], int left, int mid, int right);

/* This function sorts the input array and returns the
   number of inversions in the array */
int mergeSort(int arr[], int array_size)
{
    int *temp = (int *)malloc(sizeof(int)*array_size);
    return _mergeSort(arr, temp, 0, array_size - 1);
}

/* An auxiliary recursive function that sorts the input array and
  returns the number of inversions in the array. */
int _mergeSort(int arr[], int temp[], int left, int right)
{
  int mid, inv_count = 0;
  if (right > left)
  {
    /* Divide the array into two parts and call _mergeSortAndCountInv()
       for each of the parts */
    mid = (right + left)/2;

    /* Inversion count will be sum of inversions in left-part, right-part
      and number of inversions in merging */
    inv_count  = _mergeSort(arr, temp, left, mid);
    inv_count += _mergeSort(arr, temp, mid+1, right);

    /*Merge the two parts*/
```

```
    inv_count += merge(arr, temp, left, mid+1, right);
  }
  return inv_count;
}

/* This funt merges two sorted arrays and returns inversion count in
   the arrays.*/
int merge(int arr[], int temp[], int left, int mid, int right)
{
  int i, j, k;
  int inv_count = 0;

  i = left; /* i is index for left subarray*/
  j = mid;  /* i is index for right subarray*/
  k = left; /* i is index for resultant merged subarray*/
  while ((i <= mid - 1) && (j <= right))
  {
    if (arr[i] <= arr[j])
    {
      temp[k++] = arr[i++];
    }
    else
    {
      temp[k++] = arr[j++];

     /*this is tricky -- see above explanation/diagram for merge()*/
      inv_count = inv_count + (mid - i);
    }
  }

  /* Copy the remaining elements of left subarray
   (if there are any) to temp*/
  while (i <= mid - 1)
    temp[k++] = arr[i++];

  /* Copy the remaining elements of right subarray
   (if there are any) to temp*/
  while (j <= right)
    temp[k++] = arr[j++];

  /*Copy back the merged elements to original array*/
  for (i=left; i <= right; i++)
    arr[i] = temp[i];

  return inv_count;
}

/* Driver progra to test above functions */
int main(int argv, char** args)
{
  int arr[] = {1, 20, 6, 4, 5};
  printf(" Number of inversions are %d \n", mergeSort(arr, 5));
  getchar();
  return 0;
}
```

An explanation was given in detail here: http://www.geeksforgeeks.org/counting-inversions/

answered Mar 9 '13 at 15:46

banarun
1,558 8 29

Here is c++ solution

```
/**
*array sorting needed to verify if first arrays n'th element is greater than sencond
arrays
*some element then all elements following n will do the same
*/
#include<stdio.h>
#include<iostream>
using namespace std;
int countInversions(int array[],int size);
int merge(int arr1[],int size1,int arr2[],int size2,int[]);
int main()
{
    int array[] = {2, 4, 1, 3, 5};
    int size = sizeof(array) / sizeof(array[0]);
    int x = countInversions(array,size);
    printf("number of inversions = %d",x);
}

int countInversions(int array[],int size)
{
    if(size > 1 )
    {
    int mid = size / 2;
    int count1 = countInversions(array,mid);
    int count2 = countInversions(array+mid,size-mid);
    int temp[size];
    int count3 = merge(array,mid,array+mid,size-mid,temp);
    for(int x =0;x<size ;x++)
    {
        array[x] = temp[x];
```

```
    }
    return count1 + count2 + count3;
    }else{
        return 0;
    }
}

int merge(int arr1[],int size1,int arr2[],int size2,int temp[])
{
    int count  = 0;
    int a = 0;
    int b = 0;
    int c = 0;
    while(a < size1 && b < size2)
    {
        if(arr1[a] < arr2[b])
        {
            temp[c] = arr1[a];
            c++;
            a++;
        }else{
            temp[c] = arr2[b];
            b++;
            c++;
            count = count + size1 -a;
        }
    }

    while(a < size1)
    {
        temp[c] = arr1[a];
        c++;a++;
    }

while(b < size2)
    {
        temp[c] = arr2[b];
        c++;b++;
    }

    return count;
}
```

answered Nov 25 '14 at 18:00

Dheeraj Sachan
366 2 8

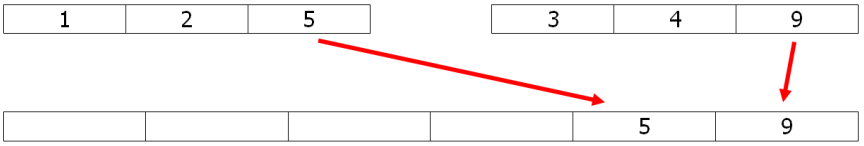
Here is O(n*log(n)) perl implementation:

```
sub sort_and_count {
    my ($arr, $n) = @_;
    return ($arr, 0) unless $n > 1;

    my $mid = $n % 2 == 1 ? ($n-1)/2 : $n/2;
    my @left = @$arr[0..$mid-1];
    my @right = @$arr[$mid..$n-1];

    my ($sleft, $x) = sort_and_count( \@left, $mid );
    my ($sright, $y) = sort_and_count( \@right, $n-$mid);
    my ($merged, $z) = merge_and_countsplitinv( $sleft, $sright, $n );

    return ($merged, $x+$y+$z);
}

sub merge_and_countsplitinv {
    my ($left, $right, $n) = @_;

    my ($l_c, $r_c) = ($#$left+1, $#$right+1);
    my ($i, $j) = (0, 0);
    my @merged;
    my $inv = 0;

    for my $k (0..$n-1) {
        if ($i<$l_c && $j<$r_c) {
            if ( $left->[$i] < $right->[$j]) {
                push @merged, $left->[$i];
                $i+=1;
            } else {
                push @merged, $right->[$j];
                $j+=1;
                $inv += $l_c - $i;
            }
        } else {
            if ($i>=$l_c) {
                push @merged, @$right[ $j..$#$right ];
            } else {
                push @merged, @$left[ $i..$#$left ];
            }
            last;
        }
    }

    return (\@merged, $inv);
}
```

answered Jan 29 at 19:03

Omid
72 8

The number of inversions can be found by analyzing the merge process in merge sort :



When copying a element from the second array to the merge array (the 9 in this exemple), it keeps its place relatively to other elements. When copying a element from the first array to the merge array (the 5 here) it is inverted with all the elements staying in the second array (2 inversions with the 3 and the 4). So a little modification of merge sort can solve the problem in O(n ln n).
For exemple, just uncomment the two # lines in the mergesort python code below to have the count.

```
def merge(l1,l2):
    l=[];#global count
    while l1 and l2:
        if l1[-1]<=l2[-1]:l.append(l2.pop())
        else:l.append(l1.pop());#count += len(l2)
    l.reverse();return l1+l2+l

def sort(l):
    t=len(l)//2
    return merge(sort(l[:t]),sort(l[t:])) if t>0 else l

count=0;print(sort([5,1,2,4,9,3]),count)
# [1, 2, 3, 4, 5, 9] 6
```

edited Apr 5 at 21:10

answered Apr 4 at 20:38

B. M.
379 1 5

The easy O(n^2) answer is to use nested for-loops and increment a counter for every inversion

```
int counter = 0;

for(int i = 0; i < n - 1; i++)
{
    for(int j = i+1; j < n; j++)
    {
        if( A[i] > A[j] )
        {
            counter++;
        }
    }
}

return counter;
```

Now I suppose you want a more efficient solution, I'll think about it.

answered Dec 3 '08 at 16:31

mbillard
18.6k 10 53 78

3 For homework questions it is best to give helpful suggestions rather than an actual solution. Teach a man to fish. – Doctor Jones Dec 3 '08 at 16:35

1 That's the obvious solution every other student will get first, I suppose their teacher wants a better implementation that will get them more points. – mbillard Dec 3 '08 at 16:37

Not necessarily, it depends upon the level of the programming course. It's not so straightforward for a beginner. – Doctor Jones Dec 3 '08 at 16:44

I recently had to do this in R:

```
inversionNumber <- function(x){
    mergeSort <- function(x){
        if(length(x) == 1){
            inv <- 0
        } else {
            n <- length(x)
            n1 <- ceiling(n/2)
            n2 <- n-n1
```

```
            y1 <- mergeSort(x[1:n1])
            y2 <- mergeSort(x[n1+1:n2])
            inv <- y1$inversions + y2$inversions
            x1 <- y1$sortedVector
            x2 <- y2$sortedVector
            i1 <- 1
            i2 <- 1
            while(i1+i2 <= n1+n2+1){
                if(i2 > n2 || i1 <= n1 && x1[i1] <= x2[i2]){
                    x[i1+i2-1] <- x1[i1]
                    i1 <- i1 + 1
                } else {
                    inv <- inv + n1 + 1 - i1
                    x[i1+i2-1] <- x2[i2]
                    i2 <- i2 + 1
                }
            }
        }
        return (list(inversions=inv,sortedVector=x))
    }
    r <- mergeSort(x)
    return (r$inversions)
}
```

answered Nov 26 '13 at 13:58

tennenrishin
647 8 18

1 @Dukeling What prompted you to withdraw your comment but not your downvote? – tennenrishin Jun 10 '14 at 17:55

Java implementation:

```
import java.lang.reflect.Array;
import java.util.Arrays;


public class main {

public static void main(String[] args) {
    int[] arr = {6, 9, 1, 14, 8, 12, 3, 2};
    System.out.println(findinversion(arr,0,arr.length-1));
}

public static int findinversion(int[] arr,int beg,int end) {
    if(beg >= end)
        return 0;

    int[] result = new int[end-beg+1];
    int index = 0;
    int mid = (beg+end)/2;
    int count = 0, leftinv,rightinv;
    //System.out.println("...."+beg+"   "+end+"  "+mid);
    leftinv = findinversion(arr, beg, mid);
    rightinv = findinversion(arr, mid+1, end);
    l1:
    for(int i = beg, j = mid+1; i<=mid || j<=end;/*index < result.length;*/ ) {
        if(i>mid) {
            for(;j<=end;j++)
                result[index++]=arr[j];
            break l1;
        }
        if(j>end) {
            for(;i<=mid;i++)
                result[index++]=arr[i];
            break l1;
        }
        if(arr[i] <= arr[j]) {
            result[index++]=arr[i];
            i++;
        } else {
            System.out.println(arr[i]+"  "+arr[j]);
            count = count+ mid-i+1;
            result[index++]=arr[j];
            j++;
        }
    }

    for(int i = 0, j=beg; i< end-beg+1; i++,j++)
        arr[j]= result[i];
    return (count+leftinv+rightinv);
    //System.out.println(Arrays.toString(arr));
}

}
```

answered Jan 12 '14 at 15:31

Anwit
9

-1 because an answer in every other language would lead to hopelessly too many answers, all of which essentially duplicate information already presented in other answers. Additionally, this is essentially a code-

only answer with no explanation, which is, at best, mainly appropriate on questions actually about that language. – Dukeling Jun 6 '14 at 16:33

1 @Dukeling Very nice welcome to the community for Anwit. His first answer gets a down vote because he tried. Very nice of you. – Venkat Sudheer Reddy Aedama Jun 6 '14 at 17:02

@VenkatSudheerReddyAedama To be fair, he's posted 6 answers already, and an answer that's not useful is not useful, regardless of how much reputation the poster has. Our voting should target content, not users. – Dukeling Jun 6 '14 at 17:07

1 @Dukeling Content is not made from ether. It comes from users. This answer may not have helped you but it definitely helps someone who is looking for an answer in Java. You down-voted my answer (stackoverflow.com/questions/337664/…) because of the same reason but I bet it would have helped if someone was looking for the same solution in Scala. If algorithm/explanation is what you care for, well there are users who care for implementation in a specific language and thats the reason you see answers in different languages. – Venkat Sudheer Reddy Aedama Jun 6 '14 at 22:41

@VenkatSudheerReddyAedama There are too many languages to keep an answer here for each one, especially considering that there are more than half a dozen approaches already presented here (it *may* have been a different story if there was absolutely only one way to do it). Too many answers dilute the answers too much - reading almost a dozen identical approaches is quite frankly a waste of time, especially when the only non-code in the answer is "Java implementation" (so I have to read the code to figure out what it's about). (And there were already two Java answers here when this was posted) – Dukeling Jun 6 '14 at 22:54

Here is my take using Scala:

```
trait MergeSort {
  def mergeSort(ls: List[Int]): List[Int] = {
    def merge(ls1: List[Int], ls2: List[Int]): List[Int] =
      (ls1, ls2) match {
        case (_, Nil) => ls1
        case (Nil, _) => ls2
        case (lowsHead :: lowsTail, highsHead :: highsTail) =>
          if (lowsHead <= highsHead) lowsHead :: merge(lowsTail, ls2)
          else highsHead :: merge(ls1, highsTail)
      }

    ls match {
      case Nil => Nil
      case head :: Nil => ls
      case _ =>
        val (lows, highs) = ls.splitAt(ls.size / 2)
        merge(mergeSort(lows), mergeSort(highs))
    }
  }
}

object InversionCounterApp extends App with MergeSort {
  @annotation.tailrec
  def calculate(list: List[Int], sortedListZippedWithIndex: List[(Int, Int)], counter: Int
= 0): Int =
    list match {
      case Nil => counter
      case head :: tail => calculate(tail, sortedListZippedWithIndex.filterNot(_._1 == 1),
counter + sortedListZippedWithIndex.find(_._1 == head).map(_._2).getOrElse(0))
    }

  val list: List[Int] = List(6, 9, 1, 14, 8, 12, 3, 2)
  val sortedListZippedWithIndex: List[(Int, Int)] = mergeSort(list).zipWithIndex
  println("inversion counter = " + calculate(list, sortedListZippedWithIndex))
  // prints: inversion counter = 28
}
```

answered Jan 27 '14 at 1:35

Venkat Sudheer Reddy A

799 4 19

-1 because an answer in every other language would lead to hopelessly too many answers, all of which essentially duplicate information already presented in other answers. Additionally, this is essentially a code-only answer with no explanation, which is, at best, mainly appropriate on questions actually about that language. – Dukeling Jun 6 '14 at 16:33

@Dukeling Have you ever heard of a concept called tail-recursion ? If you can't understand the above code you probably can't even understand the explanation with my code. I am saying this because you probably never cared to read enough. – Venkat Sudheer Reddy Aedama Jun 6 '14 at 16:59

... oh, and I find Scala to have somewhat difficult to read syntax (probably because I don't have experience in it or similar languages, but that's part of the point - this isn't a Scala question, so I shouldn't be expected to have). Tail-recursion (if that's the main / only difference from some other answers), for the most part, is an optimization, not a fundamentally change to an algorithm, i.e. not sufficient to justify a separate answer - you also didn't mention anything about tail recursion in your answer. – Dukeling Jun 6 '14 at 17:05

Wow, you down voted this answer even when you couldn't read it ? Yet managed to understand ? And figured out that it is duplicate info presented in other answers ? Very impressive. And about the tail recursion part, anyone who came here looking for implementation in Scala would know if they read the code because it's written in English. Again, I understand that the question is not about Scala but there are users/language beginners who come searching for implementation in a specific language. – Venkat Sudheer Reddy Aedama Jun 6 '14 at 22:51

It doesn't take much to spot common patterns between code samples - it's not foolproof, but it's a pretty good indication of similarity - given that it's not the only problem I have with the answer, it's not exactly a

train-smash if I get it wrong. But that doesn't mean I can actually read the code well enough to understand it. And Stack Overflow is a Q&A site, not a code repository. – Dukeling Jun 6 '14 at 23:09

I think el diablo's answer can be optimized to remove step 2b in which we delete already processed elements.

Instead we can define

# of inversion for x = position of x in sorted array - position of x in orig array

answered Feb 9 '14 at 7:05

Shireesh
19 3

Another Python solution, short one. Makes use of builtin bisect module, which provides functions to insert element into its place in sorted array and to find index of element in sorted array.

The idea is to store elements left of n-th in such array, which would allow us to easily find the number of them greater than n-th.

Complexity is O(n * log n)

```
import bisect
def solution(A):
    sorted_left = []
    res = 0
    for i in xrange(1, len(A)):
        bisect.insort_left(sorted_left, A[i-1])
        # i is also the length of sorted_left
        res += (i - bisect.bisect(sorted_left, A[i]))
    return res
```

answered Feb 16 '14 at 18:12

Tim Babych
987 9 7

`insort_left` is definitely not O(log n)... – Niklas B. Apr 21 '14 at 16:51

C code easy to understand:

```
#include<stdio.h>
#include<stdlib.h>

//To print an array
void print(int arr[],int n)
{
    int i;
    for(i=0,printf("\n");i<n;i++)
        printf("%d ",arr[i]);
    printf("\n");
}

//Merge Sort
int merge(int arr[],int left[],int right[],int l,int r)
{
    int i=0,j=0,count=0;
    while(i<l || j<r)
    {
        if(i==l)
        {
            arr[i+j]=right[j];
            j++;
        }
        else if(j==r)
        {
            arr[i+j]=left[i];
            i++;
        }
        else if(left[i]<=right[j])
        {
            arr[i+j]=left[i];
            i++;
        }
        else
        {
            arr[i+j]=right[j];
            count+=l-i;
            j++;
        }
    }
    //printf("\ncount:%d\n",count);
    return count;
}
```

```
//Inversion Finding
int inversions(int arr[],int high)
{
    if(high<1)
        return 0;

    int mid=(high+1)/2;
    int left[mid];
    int right[high-mid+1];

    int i,j;
    for(i=0;i<mid;i++)
        left[i]=arr[i];


    for(i=high-mid,j=high;j>=mid;i--,j--)
        right[i]=arr[j];

    //print(arr,high+1);
    //print(left,mid);
    //print(right,high-mid+1);

    return inversions(left,mid-1) + inversions(right,high-mid) +
merge(arr,left,right,mid,high-mid+1);

}
int main()
{
    int arr[]={6,9,1,14,8,12,3,2};
    int n=sizeof(arr)/sizeof(arr[0]);
    print(arr,n);
    printf("%d ",inversions(arr,n-1));
    return 0;
}
```

answered Jul 5 '14 at 23:07

Jerky
958 3 24

O(n log n) time, O(n) space solution in java.

A mergesort, with a tweak to preserve the number of inversions performed during the merge step.
(for a well explained mergesort take a look at
http://www.vogella.com/tutorials/JavaAlgorithmsMergesort/article.html)

Since mergesort can be made in place, the space complexity may be improved to O(1).

When using this sort, the inversions happen only in the merge step and only when we have to put an element of the second part before elements from the first half, e.g.

- 0 5 10 15

merged with

- 1 6 22

we have 3 + 2 + 0 = 5 inversions:

- 1 with {5, 10, 15}
- 6 with {10, 15}
- 22 with {}

After we have made the 5 inversions, our new merged list is 0, 1, 5, 6, 10, 15, 22

There is a demo task on Codility called ArrayInversionCount, where you can test your solution.

```
    public class FindInversions {

    public static int solution(int[] input) {
        if (input == null)
            return 0;
        int[] helper = new int[input.length];
        return mergeSort(0, input.length - 1, input, helper);
    }

    public static int mergeSort(int low, int high, int[] input, int[] helper) {
        int inversionCount = 0;
        if (low < high) {
            int medium = low + (high - low) / 2;
            inversionCount += mergeSort(low, medium, input, helper);
            inversionCount += mergeSort(medium + 1, high, input, helper);
            inversionCount += merge(low, medium, high, input, helper);
        }
        return inversionCount;
    }

    public static int merge(int low, int medium, int high, int[] input, int[] helper) {
        int inversionCount = 0;

        for (int i = low; i <= high; i++)
```

```
            helper[i] = input[i];

        int i = low;
        int j = medium + 1;
        int k = low;

        while (i <= medium && j <= high) {
            if (helper[i] <= helper[j]) {
                input[k] = helper[i];
                i++;
            } else {
                input[k] = helper[j];
                // the number of elements in the first half which the j element needs to
jump over.
                // there is an inversion between each of those elements and j.
                inversionCount += (medium + 1 - i);
                j++;
            }
            k++;
        }

        // finish writing back in the input the elements from the first part
        while (i <= medium) {
            input[k] = helper[i];
            i++;
            k++;
        }
        return inversionCount;
    }

}
```

edited Jul 12 '14 at 18:25

answered Jul 12 '14 at 18:19

Andrey Petrov
181 2 4

Here's my O(n log n) solution in Ruby:

```
def solution(t)
    sorted, inversion_count = sort_inversion_count(t)
    return inversion_count
end

def sort_inversion_count(t)
    midpoint = t.length / 2
    left_half = t[0...midpoint]
    right_half = t[midpoint..t.length]

    if midpoint == 0
        return t, 0
    end

    sorted_left_half, left_half_inversion_count = sort_inversion_count(left_half)
    sorted_right_half, right_half_inversion_count = sort_inversion_count(right_half)

    sorted = []
    inversion_count = 0
    while sorted_left_half.length > 0 or sorted_right_half.length > 0
        if sorted_left_half.empty?
            sorted.push sorted_right_half.shift
        elsif sorted_right_half.empty?
            sorted.push sorted_left_half.shift
        else
            if sorted_left_half[0] > sorted_right_half[0]
                inversion_count += sorted_left_half.length
                sorted.push sorted_right_half.shift
            else
                sorted.push sorted_left_half.shift
            end
        end
    end

    return sorted, inversion_count + left_half_inversion_count +
right_half_inversion_count
end
```

And some test cases:

```
require "minitest/autorun"

class TestCodility < Minitest::Test
    def test_given_example
        a = [-1, 6, 3, 4, 7, 4]
        assert_equal solution(a), 4
    end

    def test_empty
        a = []
        assert_equal solution(a), 0
    end

    def test_singleton
        a = [0]
        assert_equal solution(a), 0
```

```
    end

    def test_none
        a = [1,2,3,4,5,6,7]
        assert_equal solution(a), 0
    end

    def test_all
        a = [5,4,3,2,1]
        assert_equal solution(a), 10
    end

    def test_clones
        a = [4,4,4,4,4,4]
        assert_equal solution(a), 0
    end
end
```

answered Aug 5 '14 at 3:13

Brandon
137 2 12

Use mergesort, in merge step incremeant counter if the number copied to output is from right array.

answered Oct 21 '09 at 18:02

rkatiyar
162 1 9

Incrementing the counter (presumably by one) for each element is going to give you too few inversions. – Dukeling Jun 6 '14 at 15:07

Since this is an old question, I'll provide my answer in C.

```
#include <stdio.h>

int count = 0;
int inversions(int a[], int len);
void mergesort(int a[], int left, int right);
void merge(int a[], int left, int mid, int right);

int main() {
  int a[] = { 1, 5, 2, 4, 0 };
  printf("%d\n", inversions(a, 5));
}

int inversions(int a[], int len) {
  mergesort(a, 0, len - 1);
  return count;
}

void mergesort(int a[], int left, int right) {
  if (left < right) {
     int mid = (left + right) / 2;
     mergesort(a, left, mid);
     mergesort(a, mid + 1, right);
     merge(a, left, mid, right);
  }
}

void merge(int a[], int left, int mid, int right) {
  int i = left;
  int j = mid + 1;
  int k = 0;
  int b[right - left + 1];
  while (i <= mid && j <= right) {
     if (a[i] <= a[j]) {
       b[k++] = a[i++];
     } else {
       printf("right element: %d\n", a[j]);
       count += (mid - i + 1);
       printf("new count: %d\n", count);
       b[k++] = a[j++];
     }
  }
  while (i <= mid)
    b[k++] = a[i++];
  while (j <= right)
    b[k++] = a[j++];
  for (i = left, k = 0; i <= right; i++, k++) {
    a[i] = b[k];
  }
}
```

answered Nov 29 '12 at 21:27

mbreining
4,581 2 18 27

-1 because an answer in every other language would lead to hopelessly too many answers, all of which essentially duplicate information already presented in other answers. Additionally, this is essentially a code-only answer with no explanation, which is, at best, mainly appropriate on questions actually about that language. – Dukeling Jun 6 '14 at 16:32

In Java Brute force algorithm works faster than piggy backed merge sort algorithm this is because of run time optimization done by Java Dynamic compiler.

For Brute force loop rolling optimization will result in much better results.

answered Aug 21 '13 at 7:23

Yashodeep Vaidya

1

This statement doesn't mean much without proof - code to reproduce, Java version and test results. And my guess is that you tested this with fairly small arrays (assuming you tested it at all), where you'd expect the asymptotically slower algorithm with very low constant factors to outperform the asymptotically faster one with higher constant factors. – Dukeling Jun 6 '14 at 15:11

One possible solution in C++ satisfying the O(N*log(N)) time complexity requirement would be as follows.

```
#include <algorithm>

vector<int> merge(vector<int>left, vector<int>right, int &counter)
{

    vector<int> result;

    vector<int>::iterator it_l=left.begin();
    vector<int>::iterator it_r=right.begin();

    int index_left=0;

    while(it_l!=left.end() || it_r!=right.end())
    {

        // the following is true if we are finished with the left vector
        // OR if the value in the right vector is the smaller one.

        if(it_l==left.end() || (it_r!=right.end() && *it_r<*it_l) )
        {
            result.push_back(*it_r);
            it_r++;

            // increase inversion counter
            counter+=left.size()-index_left;
        }
        else
        {
            result.push_back(*it_l);
            it_l++;
            index_left++;

        }
    }

    return result;
}

vector<int> merge_sort_and_count(vector<int> A, int &counter)
{

    int N=A.size();
    if(N==1)return A;

    vector<int> left(A.begin(),A.begin()+N/2);
    vector<int> right(A.begin()+N/2,A.end());

    left=merge_sort_and_count(left,counter);
    right=merge_sort_and_count(right,counter);


    return merge(left, right, counter);

}
```

It differs from a regular merge sort only by the counter.

answered Sep 16 '13 at 1:29

oo_miguel

85 5

This seems pretty much the same as the Java and Python solutions posted earlier seemingly using the same algorithm, and thus I don't feel it adds much value beyond them. – Dukeling Jun 6 '14 at 18:12

The number of inversions in an array is half the total distance elements must be moved in order to sort the array. Therefore, it can be computed by sorting the array, maintaining the resulting permutation p[i], and then computing the sum of abs(p[i]-i)/2. This takes O(n log n) time, which is optimal.

An alternative method is given at http://mathworld.wolfram.com/PermutationInversion.html. This method is equivalent to the sum of max(0, p[i]-i), which is equal to the sum of abs(p[i]-i])/2 since the total distance elements move left is equal to the total distance elements move to the right.

EDIT: This method is wrong (see comments), and there is unfortunately no way to fix it while preserving the character of the method.

edited Nov 9 '14 at 18:37

answered Dec 3 '08 at 17:36

Geoffrey Irving
1,582 13 27

Why the downvote? – Geoffrey Irving Nov 6 '14 at 19:08

The first statement is incorrect. For example, for {3,2,1} the correct answer is 3 (all pairs out of order), but the sum of distances is 4, hence your algorithm returns 2. – Eyal Schneider Nov 8 '14 at 19:45

Indeed, and the other part is wrong too (it also gives 2). I've confirmed that there's no way to fix this method. That is, there is no f(p[i]-i) such that the above sum works. Alas. – Geoffrey Irving Nov 9 '14 at 18:35

protected by Community ♦ May 12 '14 at 12:10

Thank you for your interest in this question. Because it has attracted low-quality answers, posting an answer now requires 10 reputation on this site.

Would you like to answer one of these unanswered questions instead?